

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

R.HARIKA(1925721)

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Constraint-Based Problem Solving (High) Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Given a complex SQL query with nested joins and sub-queries, draw the query execution plan and apply heuristic optimization rules step-by-step.	BL4 – Analyze	5
2	Apply cost-based optimization to choose between nested-loop join, sort-merge join, and hash join for the given relations with specified tuple counts and page sizes.	BL4 – Analyze	5
3	Implement JDBC connectivity in Java to a MySQL database and write code to execute parameterized queries with PreparedStatement.	BL3 – Apply	5
4	Given a schedule of four concurrent transactions, determine whether it is conflict-serializable using a precedence graph.	BL4 – Analyze	5
5	Demonstrate the application of Basic Two-Phase Locking (2PL) on a given transaction set. Identify and resolve any deadlocks.	BL3 – Apply	5

6	Apply the Wait-Die and Wound-Wait timestamp-based deadlock prevention schemes on a given set of transactions requesting locks.	BL4 – Analyze	5
7	Given transaction logs with checkpoints, apply the Immediate Update recovery protocol to recover the database after a system failure.	BL3 – Apply	5
8	Apply Deferred Update (NO-UNDO/REDO) recovery technique on a given log file with committed and uncommitted transactions.	BL3 – Apply	5
9	Design a concurrency control mechanism using strict 2PL for a banking system where T1 transfers funds and T2 reads balance simultaneously.	BL4 – Analyze	5
10	Write pseudocode for query optimization using heuristics: push down selection, project early, and order joins by smaller intermediate results.	BL4 – Analyze	5

Code of Conduct:

I RASINENI HARIKA(192572140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Harika

Constraint-Based Problem Solving Assessment Rubric					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization

Signature of the student:

RUBRICS FOR CALCULATION:

Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Query Execution Plan and Heuristic Optimization

Heuristic optimization improves a query by applying rules that reduce the size of intermediate relations.

Example query:

```
SELECT E.name, D.dname
FROM Employee E
JOIN Department D ON E.dept_id = D.dept_id
WHERE E.salary > 50000
AND D.location = 'Chennai';
```

Initial query tree:

```

      PROJECT name, dname
        |
      JOIN
    /      \
Employee  Department
```

Step 1 – Push selections down:

```

      PROJECT name,dname
        |
      JOIN
    /      \
SELECT salary>50000  SELECT location='Chennai'
    |              |
Employee          Department
```

Step 2 – Push projections down:

Keep only attributes required for selection/join/output.

```

      PROJECT name,dname
        |
```

```

      JOIN
    /   \
PROJECT  PROJECT
name,dept_id,salary dept_id,dname,location

```

Step 3 – Choose join order: Perform the join using the smaller filtered relations first.

Result: Fewer tuples are processed, reducing CPU, memory, and disk I/O cost.

2. Cost-Based Selection of Join Algorithms

Suppose relations **R** and **S** have:

- R = 1,000 tuples, 100 pages
- S = 5,000 tuples, 500 pages
- Available memory = 20 pages

Nested-Loop Join:

Approximate block nested-loop cost:

Sort-Merge Join:

Requires sorting both relations and then merging. Its cost is approximately:

It is relatively expensive because both relations need sorting.

Hash Join:

If the smaller relation can be partitioned efficiently, the approximate cost is:

Choice: Hash join is preferred because it has the lowest estimated I/O cost for an equality join.

Conclusion: Cost-based optimization estimates the cost of each algorithm and selects the cheapest feasible one.

3. JDBC Connectivity Using PreparedStatement

Steps:

1. Load/connect to MySQL.
2. Create a Connection.
3. Create PreparedStatement.
4. Set parameters.
5. Execute the query.
6. Process results.
7. Close resources.

```
import java.sql.*;
```

```

public class JDBCExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/company";
        String user = "root";
        String password = "password";

        String sql = "SELECT * FROM Employee WHERE dept_id = ?";
    }
}

```

```

try (Connection con = DriverManager.getConnection(url, user, password);
    PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setInt(1, 10);

    ResultSet rs = ps.executeQuery();

    while (rs.next()) {
        System.out.println(rs.getInt("id") + " "
            + rs.getString("name"));
    }

} catch (SQLException e) {
    e.printStackTrace();
}
}

```

Advantages of PreparedStatement:

- Prevents SQL injection.
- Supports parameterized queries.
- Improves readability.
- Can improve performance for repeated execution.

4. Conflict Serializability Using Precedence Graph

Consider:

T1: R(A) W(B)

T2: W(A) R(B)

T3: R(B) W(C)

T4: R(C) W(A)

Steps:

1. Create one node for every transaction: T1, T2, T3, T4.
2. For every conflicting operation on the same data item, add an edge from the earlier transaction to the later transaction.

Examples:

- T1:R(A) before T2:W(A) → **T1 → T2**
- T1:W(B) before T2:R(B) → **T1 → T2**
- T2:R(B) before T3:W(B) → **T2 → T3**
- T3:W(C) before T4:R(C) → **T3 → T4**
- T2:W(A) before T4:W(A) → **T2 → T4**

If the resulting graph contains **no cycle**, the schedule is **conflict-serializable**.

If a cycle exists, such as:

T1 → T2 → T3 → T1

the schedule is **not conflict-serializable**.

5. Basic Two-Phase Locking (2PL)

2PL divides transaction execution into two phases:

1. **Growing phase:** Transaction can acquire locks but cannot release them.
2. **Shrinking phase:** Transaction can release locks but cannot acquire new locks.

Example:

T1: LOCK-X(A)
 LOCK-X(B)
 READ/WRITE A,B
 UNLOCK(A)
 UNLOCK(B)

T1 first obtains all required locks and then releases them.

Suppose:

T1 holds A and waits for B

T2 holds B and waits for A

This creates:

T1 → T2

T2 → T1

Therefore, a **deadlock** occurs.

Resolution:

- Detect the cycle.
- Abort one transaction, e.g., T2.
- Release T2's locks.
- Allow T1 to continue.
- Restart T2 later.

Thus, 2PL ensures conflict serializability, but basic 2PL can produce deadlocks.

6. Wait-Die and Wound-Wait Schemes

Assume:

T1 timestamp = 5 (older)

T2 timestamp = 10 (younger)

T2 holds a lock requested by T1.

Wait-Die

- Older transaction requests lock held by younger → **older waits**.
- Younger requests lock held by older → **younger aborts (dies)**.

Therefore:

T1 requests T2's lock → T1 waits

T2 requests T1's lock → T2 aborts

Wound-Wait

- Older transaction requests lock held by younger → **younger is aborted (wounded)**.
- Younger requests lock held by older → **younger waits**.

Therefore:

T1 requests T2's lock → T2 aborts

T2 requests T1's lock → T2 waits

Difference:

Scheme Older requests younger Younger requests older

Wait-Die Wait Abort

Wound-Wait Abort younger Wait

Both schemes prevent deadlocks by ensuring cycles cannot form.

7. Immediate Update Recovery

In **Immediate Update**, database changes can be written to disk before a transaction commits.

Suppose the log is:

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<T1 COMMIT>

<CHECKPOINT>

<T2, C, 300, 350>

<T2 CRASH>

After a system failure:

- **Committed transactions → REDO**
- **Uncommitted transactions → UNDO**

Here:

T1 → committed → REDO

T2 → uncommitted → UNDO

Therefore:

REDO T1 changes

UNDO T2 changes

Checkpoint reduces the amount of log that must be examined during recovery.

Result: Database is restored to a consistent state.

8. Deferred Update – NO-UNDO/REDO

In **Deferred Update**, database modifications are not written to disk until the transaction commits.

Example:

<T1 START>

<T1, A, 100, 150>

<T2 START>

<T2, B, 200, 250>

<T1 COMMIT>

<T2, C, 300, 350>

SYSTEM CRASH

After crash:

- T1 is **committed** → **REDO T1**
- T2 is **uncommitted** → **ignore T2**

Committed transactions → REDO

Uncommitted transactions → NO action

There is **no UNDO** because uncommitted updates have not reached the database.

Hence it is called **NO-UNDO/REDO recovery**.

9. Strict 2PL for Banking System

Suppose:

- **T1:** Transfers ₹1,000 from Account A to B.
- **T2:** Reads the balance of A.

T1 uses exclusive locks:

T1:

LOCK-X(A)

LOCK-X(B)

$A = A - 1000$

$B = B + 1000$

COMMIT

UNLOCK(A)

UNLOCK(B)

T2 needs a shared lock:

T2:

LOCK-S(A)

READ A

COMMIT

UNLOCK(A)

If T2 requests A while T1 is updating it:

T1: X-lock(A)

T2: waits for A

After T1 commits:

T1: unlock(A)

T2: obtains S-lock(A)

T2: reads updated balance

Benefits of strict 2PL:

- Prevents dirty reads.
- Prevents cascading rollback.
- Maintains consistency.
- Produces conflict-serializable schedules.

10. Pseudocode for Heuristic Query Optimization

OPTIMIZE_QUERY(Q):

1. Convert Q into a relational algebra/query tree.

2. Push selections down:
For every selection σ :
Move σ as close as possible to base relations.

3. Push projections down:
For every relation:
Keep only attributes required by
SELECT, JOIN, WHERE and GROUP BY.

4. Simplify the query tree:
Remove unnecessary selections and projections.

5. Determine join order:
Estimate intermediate relation sizes.

Join the smallest/selectively filtered
relations first.

6. Select suitable join algorithms
based on relation sizes and available memory.

7. Return the optimized query execution plan.

Main heuristic rules:

1. **Push selections down** → reduces tuples early.
2. **Project early** → reduces the number of attributes and data size.
3. **Order joins intelligently** → produces smaller intermediate relations.